# SmartDock: Speaker Script for the Presentation

**Presenter:** Ramaguru S (227003181) · **Guide:** Dr. ALAMELU R

**Slides:** `docs/slides/slides.pdf` (32 slides) · **Live site:** `https://smartdock.onrender.com`

---

## Before you start (5 minutes before)

1. Open `https://smartdock.onrender.com` and wait until the login page appears. The free server sleeps and needs up to a minute to wake up.
2. Log in as **admin@smartdock.demo / Demo@123**, go to **Scheduling rules → Reset demo data**, confirm, and log in again. This makes "today" full of live trucks.
3. Keep two browser tabs ready: one logged in as the **manager**, one in a private window as the **carrier** (carrier@smartdock.demo / Demo@123).
4. Have the slides open in full-screen.

**Suggested timing (about 15–18 minutes + questions):**

Slides 1–9 → 5 min · Slides 10–19 → 5 min · Slides 20–24 + live demo → 5 min · Slides 25–32 → 3 min.

If time is short, skip slides 13, 15 and 31 (DFD level 1, ER diagram, references). Just say "the detailed diagrams are in the report".

**One sentence to remember for the whole presentation:**

> *"SmartDock is like a doctor's appointment system, but for trucks at a warehouse: carriers book a time slot at the right loading dock, and the warehouse runs its yard to that plan and measures everything."*

---

## Slide 1: Title

**On screen:** Project name, your details, guide, programme, academic year.

**What to say:**

> "Good morning. I am Ramaguru S, register number 227003181, from the Department of Computer Science and Engineering. My project, under the guidance of Dr. Alamelu R, is called **SmartDock**: a dock appointment scheduling and yard management system for warehouses and distribution centres.
>
> In one line: SmartDock lets transport companies book a time slot to load or unload at a warehouse, much like booking a doctor's appointment, and gives the warehouse full control and data about every truck that comes in."

**Stress:** The simple analogy (appointments for trucks).

---

## Slide 2: Outline

**On screen:** The 18 topics.

**What to say:**

> "I'll first explain the supply-chain problem and why it matters, then my objectives and the existing systems. After that I'll show the design, how it was built, a live demo of the deployed system, how I tested it, and the results, including a before-and-after comparison."

**Tip:** Don't read the list. Spend at most 15 seconds here.

---

### Slide 3: Introduction: Role of IT in Modern Supply Chains

**On screen:** Visibility, synchronisation, automation, measurement; the chain Supplier → Transport → **Dock & yard** → Warehouse → Customer.

**What to say:**

> "A supply chain is a chain of independent companies: suppliers, transporters, warehouses and customers. Today, companies don't compete only on product. They compete on how well this whole chain is **coordinated**.
>
> Information technology makes that coordination possible in four ways:
>
> - **visibility**: everyone sees the same real-time picture;
> - **synchronisation**: everyone works to one plan;
> - **automation**: software rules replace phone calls;
> - **measurement**: every event becomes data for KPIs.
>
> Big companies already use ERP, warehouse management and transport management systems. But look at the highlighted box: the **loading dock**, where the truck physically meets the warehouse. It is the hand-over point between transport and warehousing, and in many warehouses it is still managed by phone calls and a whiteboard. That gap is where my project sits."

**Stress:** The dock is the hand-over point, and it's usually not digitised.

---

### Slide 4: Problem Statement

**On screen:** Problems without appointments; the boxed problem statement.

**What to say:**

> "When there are no appointments, every carrier wants to come at the same convenient times, usually mid-morning and mid-afternoon. So we get:
>
> - **queues of trucks** in the yard and even on the road, while docks sit idle at other hours;
> - **detention**: drivers wait unpaid for hours, and the warehouse can't plan its labour;
> - **double-booking**: two trucks promised the same door, or a refrigerated or hazardous load sent to a door that can't handle it;
> - **no accountability**: nobody records who came late or never came;
> - **no data**: the manager can't answer 'how busy are my docks?' or 'what is my truck turnaround time?'

So my problem statement is: build a web platform where warehouses publish their dock capacity, carriers book **compatible, conflict-free** slots themselves, and every truck movement is captured to measure **utilisation, punctuality and turnaround**, securely, for many warehouses and carriers at once."

**Stress:** Read the boxed statement slowly. It is the heart of the project.

---

### Slide 5: Objectives

**On screen:** Nine objectives.

**What to say:**

"From that problem I set nine objectives. The most important ones are:

- **Multi-tenant**: many warehouses and many carriers on one platform, each seeing only their own data.
- **Configurable scheduling**: each warehouse sets its own hours, slot length of 15 to 90 minutes, a buffer between trucks, holidays and dock maintenance.
- **Matching loads to docks**: a refrigerated truck should only see refrigerated docks, and a hazardous load only hazmat-certified docks.
- **A guarantee that a slot is never overbooked**, even if many people click 'book' at the same moment. I'll show later that this is harder than it sounds.
- Real-world features: **weekly recurring bookings, rescheduling and a waitlist**.
- Finally, **digitising the yard**: gate check-in, dock start and departure; detecting late arrivals and no-shows; scoring carriers; and producing KPIs, an audit trail and notifications. All of it deployed on the cloud and tested."

---

### Slide 6: Scope and Limitations

**On screen:** Scope vs limitations.

**What to say:**

"The **scope** covers three kinds of users: warehouse managers, carriers and a super administrator. It covers the complete appointment life cycle, the waitlist, yard events, KPIs, the audit log, notifications and exports. It runs as one cloud service that works in any browser, and comes with a realistic demo dataset.

I want to be honest about the **limitations**:

- it is not yet connected to an ERP, so purchase-order and shipment numbers are typed in;
- gate check-in is a button click, not automatic number-plate recognition;
- every slot at a site has the same length;
- the super-admin role exists for security but doesn't have its own screens yet;
- the evaluation uses generated and simulated data, not a live warehouse;

> - it runs on a free cloud plan, so the first request after idle time is slow.
>
> These limitations are also my future-work list."

**Stress:** Being upfront about limitations builds credibility with examiners.

---

### Slide 7: Background: Docks, Yards and Dock KPIs

**On screen:** Definitions and four KPIs.

**What to say:**

> "A few terms the whole project uses:
>
> - A **dock door** is the bay where a truck backs in. **Inbound** docks receive goods from suppliers; **outbound** docks ship goods to customers.
> - Trucks come in different **equipment types**. A **reefer** is a refrigerated trailer for food or medicine and needs a cold-chain dock. A **flatbed** carries steel or machinery. **Hazmat**, meaning hazardous materials like chemicals, needs a certified bay.
> - The **yard** is the parking area where trucks wait.
> - An **appointment** is a reserved door for a time window.
>
> Warehouses measure dock performance with four KPIs: **dock utilisation** (how much of the offered capacity is used), **on-time arrival rate**, **no-show rate**, and **turnaround time**, which is yard waiting time plus loading time. SmartDock calculates all four automatically."

---

### Slide 8: Literature Survey and Existing Systems

**On screen:** Research bullets and the comparison table.

**What to say:**

> "The best-studied version of this idea is the **truck appointment system at sea ports**. Studies at the ports of Los Angeles and Long Beach found that appointments reduce congestion and emissions, but **only when the system really controls arrivals**. Other papers modelled how limiting trucks per time window cuts turn times. Warehouse research identifies receiving and shipping as critical planning problems.
>
> One more reference matters to my design: Kleppmann's work on databases describes **'write skew'**, a bug where two people book the last place at the same instant. I designed against exactly that.
>
> The table compares the options. Phone and spreadsheets cost nothing but give no control or data. Port systems are built for container terminals. Commercial scheduling software and big warehouse suites work but are subscription-based or expensive. SmartDock combines self-service, guaranteed capacity, dock-type matching, waitlist and analytics, and it is free and open source."

**Tip:** Point at the last column of the table while speaking.

---

### Slide 9: Gap and Proposed Solution

**On screen:** Gap vs SmartDock.

**What to say:**

> "So the **gap** is: small and medium warehouses are stuck between manual methods and very expensive enterprise software. Port systems don't fit warehouses with mixed docks. Simple calendar tools ignore dock compatibility and overbook at peak times. And many tools separate booking from what really happens in the yard, so there are no KPIs and no accountability.
>
> **SmartDock fills this gap**: every warehouse sets its own rules and approves its own carriers; carriers see only compatible docks and valid slots; capacity is guaranteed; the full journey is tracked from request through approval, gate, dock and departure; and everything turns into KPIs, a reliability score, reports and an audit trail. It is free and deploys from one configuration file."

---

### Slide 10: System Architecture

**On screen:** Users → React web app → Express API (security, controllers, services, data access) → MongoDB Atlas; GitHub/Render CI; e-mail/SMS.

**What to say:**

> "This is the architecture, in three layers.
>
> **Top:** the users (warehouse manager, carrier and gate staff) use a **React web application** in their browser. No installation is needed.
>
> **Middle:** every request goes to my **Node.js/Express server**, and passes through four layers in order:
>
> 1. **Security**: a rate limiter, login token check, role check and a tenant check, so a carrier can only touch warehouses that approved it.
> 2. **Controllers**: the API endpoints for bookings, docks, analytics and so on.
> 3. **Services**: the business logic: the scheduling engine, waitlist, reliability scoring, notifications, audit and background jobs.
> 4. **Data access**: database models with validation.
>
> **Bottom: MongoDB Atlas**, a cloud database. Importantly, it supports **transactions**, which I need for the booking guarantee. Optional e-mail and SMS providers send notifications. Whenever I push code to **GitHub**, **Render** automatically rebuilds and redeploys the live site."

**Stress:** One cloud service + one cloud database; automatic deployment.

---

### Slide 11: Context Diagram (DFD Level 0)

**On screen:** SmartDock in the centre with Carrier, Warehouse manager, Gate staff and E-mail/SMS provider.

**What to say:**

> "This context diagram shows SmartDock as one process and who it talks to.

- The **carrier** sends access requests, bookings, reschedules and waitlist requests, and receives slots, status updates, its score and notifications.
- The **warehouse manager** sends dock settings, rules, approvals and yard events, and receives the schedule, KPIs, reports and the audit log.
- **Gate staff** record check-in and departure and get the printed gate sheet.
- Messages go out through the **e-mail and SMS provider**."

---

### Slide 12: Appointment Lifecycle (State Machine)

**On screen:** PENDING → APPROVED → CHECKED_IN → IN_PROGRESS → COMPLETED, plus REJECTED, CANCELLED, NO_SHOW.

**What to say:**

"Every appointment moves through these states, exactly like a parcel being tracked.

It starts **Pending** when the carrier requests it. The manager **approves** or **rejects** it. When the truck reaches the gate it is **checked in**, and at that moment the system records whether it is **on time or late**. When loading starts it is **in progress** at the dock, and when the truck leaves it is **completed**.

If an approved truck never arrives, the system automatically marks it **no-show** after a grace period.

One important design rule: **completed, cancelled, rejected and no-show are final**. A cancelled booking can never be revived, because its slot may already have been given to someone else."

---

### Slide 13: Data Flow Diagram (Level 1)

**On screen:** Six processes and five data stores.

**What to say (short):**

"Level 1 breaks the system into six processes: **manage access**, **configure site and docks**, **schedule appointments**, **execute the yard**, **notify and audit**, and **analyse performance**. They share five data stores: users and access, warehouses and docks, bookings, the waitlist, and the audit and notification logs.

The key flow is in the middle: rules and approved access feed scheduling; scheduling writes bookings; yard execution updates them and produces events; and analytics reads the bookings and timestamps to produce KPIs."

---

### Slide 14: Use Case Diagram

**On screen:** Carrier use cases (left), manager use cases (right), scheduler at the bottom.

**What to say:**

> "On the left is what a **carrier** can do: register and request warehouse access, book a compatible slot, book a weekly series, reschedule or cancel, join a waitlist, and view its score and notifications.
>
> On the right, the **warehouse manager**: configure docks, hours and holidays; approve carriers; approve or reject appointments; record gate check-in, dock start and completion; view KPIs and export reports; and review the audit log.
>
> At the bottom, the **scheduler**, meaning automatic background jobs that run every minute: marking no-shows, expiring old requests, sending reminders and assigning waitlisted trucks."

---

### Slide 15: Entity–Relationship Diagram

**On screen:** USER, WAREHOUSE, DOCK, BOOKING, VENDOR_ACCESS, WAITLIST_ENTRY, AUDIT_LOG, NOTIFICATION.

**What to say:**

> "These are the eight data collections.
>
> A **warehouse** has many **docks**; each dock stores its direction, trailer types, hazmat flag, capacity and maintenance windows.
>
> A **user** is a manager or a carrier. **Vendor access** connects carriers to warehouses with a status (pending, approved or rejected), which is how one carrier can work with several warehouses.
>
> **Booking** is the central record: dock, carrier, time, load details, purchase order number, pallets, every yard timestamp, and the history of reschedules.
>
> **Waitlist entries**, **audit logs** and **notifications** complete the model."

---

### Slide 16: Technology Stack

**On screen:** Table of technologies.

**What to say:**

> "The whole project is written in **TypeScript**, a safer version of JavaScript, on both front end and back end.
>
> - **Front end:** React with Vite and Tailwind CSS for the interface, React Query for live data refresh, and jsPDF for PDF reports.
> - **Back end:** Node.js and Express, with Zod validating every request, JWT tokens for login, and bcrypt for password hashing.
> - **Database:** MongoDB Atlas, using transactions, indexes and aggregation queries for analytics.
> - **Cloud:** Render hosts the application and redeploys automatically from GitHub.
> - **Testing:** an automated suite runs a temporary database and two servers to check 58 scenarios.
>
> This is the MERN stack: MongoDB, Express, React, Node."

---

### Slide 17: Modules

**On screen:** Ten modules.

**What to say:**

> "Functionally the system has ten modules. Let me group them:
>
> - **Setup:** identity and access (who can do what) and site and dock configuration.
> - **Planning:** the scheduling engine, booking with weekly series and rescheduling, and the waitlist.
> - **Execution:** the yard module: gate check-in, dock start, completion and no-shows.
> - **Insight and control:** the carrier reliability score, analytics and reports, notifications, and the audit trail.
>
> Together they cover plan, execute and measure, the full cycle of dock operations."

**Stress:** Plan → Execute → Measure.

---

### Slide 18: Implementation: Key Logic

**On screen:** Slot formula, compatibility rule, arrival rule, background jobs, reliability formula, waitlist.

**What to say:**

> "Four pieces of logic are worth explaining.
>
> **One, the slot grid.** If a warehouse opens at 8:00 with 30-minute slots and a 15-minute buffer, the system generates 8:00 to 8:30, then 8:45 to 9:15, and so on. It calculates these in the **warehouse's own time zone**, so daylight-saving changes and different countries are handled correctly.
>
> **Two, compatibility.** A dock is offered only if its direction matches, it supports the trailer type, and it is hazmat-certified when the load is hazardous.
>
> **Three, arrivals.** At gate check-in, if the truck is more than 15 minutes after its slot start, it is marked **late**.
>
> **Four, the reliability score**, on the right: on-time arrivals count fully, late arrivals count half, and no-shows count zero, divided by all trucks that were due. That gives a score out of 100 and a tier from A (excellent) to D (poor).
>
> And the **waitlist**: when someone cancels, the first compatible waiting carrier is booked automatically."

**Tip:** Give the score example: *"10 trips, 8 on time, 1 late, 1 no-show → (8 + 0.5) ÷ 10 = 85 → tier B."*

---

### Slide 19: Implementation: Concurrency-Safe Capacity Reservation

**On screen:** Flowchart + explanation of write skew.

**What to say:**

> "This is the most important technical part. Imagine a dock that takes **one** truck at 10 o'clock, and two carriers click 'Book' at the **same millisecond**.
>
> A simple program checks 'is there space? yes' for both of them, then saves both, and the slot is double-booked. This is called **write skew**. Surprisingly, even wrapping it in a database transaction doesn't help, because each transaction looks at its own snapshot of the data.
>
> My solution, shown in the flowchart: every booking first **writes to the dock's own record**, like taking a lock on the door. Now the two requests collide on the same record, the database rejects one and automatically retries it, and the retry sees the slot is full and returns 'slot full'. I also added a lock inside each server.
>
> This works even when the website runs on **several servers at once**, and the same routine protects normal bookings, recurring series, reschedules and the waitlist. I'll show the proof in the testing slide."

**Stress:** "Even a transaction alone does not help." That is the insight examiners remember.

---

### Slide 20: Screens: Yard Schedule (Warehouse Manager)

**On screen:** Screenshot of the yard schedule.

**What to say:**

> "This is the manager's daily **control tower**. For the selected day, each row is one truck: time slot, dock, carrier, purchase order number, truck and driver, and whether it's inbound or outbound with what equipment.
>
> On the right is the **next action**: Approve for new requests, **Gate check-in** when the truck arrives, Start at dock, and Complete. The manager can also move a booking to another slot or mark a no-show. The page refreshes itself every 10 seconds and can be exported as CSV or PDF, or printed as a **gate sheet** for security staff."

---

### Slide 21: Screens: Docks and Carrier Scorecard

**On screen:** Dock cards; carrier scorecard.

**What to say:**

> "On the left are the **dock doors**. Each one shows its direction, the trailer types it supports, whether it's hazmat-certified, and how many trucks it handles per slot. Door D-05 shows a **planned maintenance window**; during that time its slots can't be booked.
>
> On the right is the **carrier scorecard**: on-time percentage, late arrivals, no-shows and average delay, with the reliability score and tier. You can see Swift Freight at 93 (tier A) and Metro Haulers at 61 (tier C). The manager also approves new carriers here and can revoke access."

---

### Slide 22: Screens: Analytics, Rules and Audit

**On screen:** KPI dashboard, heatmap, activity log.

**What to say:**

> "Left: the **KPI dashboard**: appointments, dock utilisation, on-time rate, no-show rate, turnaround, yard wait, dock time and pallets handled, plus daily volume and busiest hours.
>
> Middle: the **demand heatmap**. The darker or brighter a cell, the more trucks arrive on that weekday and hour. The manager immediately sees the morning and afternoon peaks, and below it the utilisation of each dock.
>
> Right: the **activity log**, a permanent record of who did what and when, including automatic actions such as 'waitlist auto-assigned'. This is useful for disputes, for example when a carrier claims it was on time."

---

### Slide 23: Screens: Carrier Booking Wizard

**On screen:** Slot step, details step, series result.

**What to say:**

> "This is the carrier's side. After choosing the warehouse and load type, here a refrigerated inbound load, the carrier sees **only the reefer dock**. Green slots are free; red slots are full and offer **'join waitlist'**.
>
> Next the carrier enters truck number, driver, phone, purchase order number and pallets, and can tick **'repeat every week'**.
>
> On the right is the result of a four-week series: **three weeks booked, one week skipped because it was full**. The system books what it can and clearly reports what it couldn't, instead of failing completely."

---

### Slide 24: Live Demo

**On screen:** Links, logins and the demo plan.

**What to say:**

> "Now let me show it working live on the cloud." *(Switch to the browser and follow the demo script below. Keep it to 4–5 minutes.)*

---

### Slide 25: Testing and Validation

**On screen:** Test approach; double-booking experiment table; live response times.

**What to say:**

> "I tested the system in three ways.
>
> First, an **automated suite of 58 checks**. It starts a temporary database and **two separate servers**, like a real cloud deployment, and tests security, overbooking, slot rules, dock matching, maintenance, holidays, the booking life cycle, series, waitlist, rescheduling, reliability scores, audit and analytics. **All 58 pass.**

> Second, the **double-booking experiment** in this table. I sent **40 booking requests at the same moment** to a slot that holds one truck. With the protection switched off, **15 trucks got in**, which is 14 too many. With SmartDock's algorithm, **exactly 1**. For a three-truck slot: 13 versus exactly 3.
>
> Third, I measured the live site: about 1.3 seconds for slot availability and 1.6 seconds for the yard schedule, on the free plan, with the server in the USA and the database in India."

**Stress:** "15 versus 1" is your strongest result. Say it clearly.

---

### Slide 26: Results: KPIs from the Deployed System

**On screen:** KPI table + carrier reliability chart.

**What to say:**

> "These are the numbers the live system reports for the Chennai demo warehouse over 30 days. To be clear, the demo data is generated to behave like a real warehouse.
>
> 910 appointments; **40% dock utilisation**; **78.6% on time**; **8.6% no-shows**; average turnaround **57 minutes**, made up of 17 minutes waiting and 40 minutes loading; and over 15,000 pallets handled. The busiest hours are 10 to 11 and 3 to 4 in the afternoon.
>
> What does this tell a manager? Utilisation is only 40%, but almost 9% of booked trucks never come, so capacity is wasted. That is exactly why the **waitlist** and the **reliability score** exist. The chart shows the score clearly separating good carriers (93) from poor ones (61), which is useful when negotiating contracts."

---

### Slide 27: Results: Before vs. After (Simulation)

**On screen:** Two charts + comparison table.

**What to say:**

> "To show the impact, I built a **simulation**: one warehouse, 6 docks, 60 trucks a day, run for 60 days. Same docks, same work. The only difference is whether trucks have appointments.
>
> **Without appointments** (orange), trucks pile up in the morning and afternoon peaks. The yard queue reaches about **7 trucks**, the average wait is **19.4 minutes**, and **27.7% of trucks wait more than half an hour**.
>
> **With SmartDock** (blue), arrivals are spread across the day: the average wait drops to **about 1 minute**, only **0.3%** wait more than 30 minutes, and the queue almost disappears.
>
> This is a simplified model, so the real gain depends on carrier discipline, but it shows the order of magnitude, and it matches what port studies report."

**Stress:** Point at the orange peak, then the flat blue line.

---

### Slide 28: Benefits and Real-World Applications

**On screen:** Benefits list + applications box.

**What to say:**

"In supply-chain terms the benefits are:

- **Cost**: less detention and overtime, with labour planned to the known workload.
- **Visibility**: both sides see the same live schedule.
- **Speed**: self-service booking, and yard waiting falls from about 19 minutes to about 1.
- **Traceability**: every truck has a purchase order reference, timestamps and an audit trail.
- **Reliability and safety**: no double-booking, hazmat goes only to certified bays, and carriers are accountable.
- **Sustainability**: shorter queues mean less engine idling and pollution.

It applies to retail and e-commerce distribution centres, cold-chain and pharma warehouses, chemical depots, third-party logistics companies running many client warehouses, manufacturing plants and container freight stations."

---

### Slide 29: Conclusion

**On screen:** Four conclusion points.

**What to say:**

"To conclude: I designed, built, tested and deployed a **multi-tenant dock appointment and yard management system**. All objectives were met: configurable scheduling, dock matching, **guaranteed capacity**, recurring bookings, rescheduling, waitlist, yard tracking, reliability scores, KPIs, audit and notifications. It is verified by 58 automated checks and the concurrency experiment, and it is **live on the cloud right now**.

The main message: a focused piece of IT at the point where transport meets the warehouse turns a phone-based, invisible process into a **measured, collaborative** one."

---

### Slide 30: Future Enhancements

**On screen:** AI/ML, optimisation, IoT, integration, mobile, scale.

**What to say:**

"Next steps:

- **AI/ML**: predict which carriers are likely to be late or not show up, and safely overbook those slots a little; forecast daily volume for staffing.
- **Optimisation**: automatically assign the best dock to each truck, and vary slot length by load size.
- **IoT**: automatic gate check-in with number-plate cameras or RFID, sensors at dock doors, and temperature monitoring for reefers.
- **Integration**: pull purchase orders and shipment notices from ERP and warehouse systems.

> - **Mobile**: a driver app with directions to the assigned door.
> - **Scale**: real-time updates, an Asian server region for speed, and a super-admin console for logistics companies."

---

### Slide 31: References

**On screen:** 11 references.

**What to say:**

> "These are my main references: supply-chain textbooks by Chopra and Christopher, research on port truck appointment systems and warehouse operations, Law's book on simulation, Kleppmann's book for the concurrency problem, and the official MongoDB, OWASP and framework documentation."

**Tip:** Five seconds, then move on.

---

### Slide 32: Thank You

**What to say:**

> "Thank you. The system is live at smartdock.onrender.com and the full source code is on GitHub. I'm happy to take your questions."

---

## Live demo script (4–5 minutes)

Do it in this order; each step shows one supply-chain idea.

**1. Carrier books (carrier tab), about 1.5 min**

- Log in as **carrier@smartdock.demo**. *"This is Swift Freight Lines, a transport company. It has a reliability score of 93, tier A."*
- Click **+ Book dock appointment** → choose **Chennai Central Distribution Centre** → **Inbound** → trailer **Reefer**.
- *"Notice that only the cold-chain dock D-02 is offered, because the system matches the truck to compatible docks."*
- Pick tomorrow → point at a red slot: *"Full. I could join the waitlist."* → pick a green slot.
- Fill truck **TN 22 AB 1234**, driver name and phone, PO **PO-4512277**, 18 pallets → tick **Repeat every week, 4 weeks** → **Request appointment**.
- *"It booked the weeks that were free and tells me which week was skipped."*

**2. Manager approves and runs the yard (manager tab), about 1.5 min**

- Log in as **admin@smartdock.demo**, click the **bell**: *"The manager was notified immediately."*
- **Yard schedule** → click tomorrow on the calendar → find the new truck → **Approve**.
- Go back to **today** → pick a confirmed truck → **Gate check-in**: *"It records the time and whether the truck is on time or late."* → **Start at dock** → **Complete & depart**: *"Now we have its turnaround time."*
- Click **Print gate sheet**: *"This is what security staff get at the gate."*

**3. Control and insight, about 1.5 min**

- **Docks & maintenance**: *"Each door has a type; door D-05 has planned maintenance, so its slots are blocked."*
- **Analytics & KPIs**: point at utilisation, on-time rate, no-show rate, turnaround and the heatmap → click **Export PDF report**.
- **Activity log**: *"Every action, including the ones I just did, is recorded with who and when."*

**4. Optional, if time remains**

- **Carriers & waitlist** → approve **Nova Movers**: *"New carriers must be approved by each warehouse before they can book."*
- **Scheduling rules** → change slot length to 30 minutes and show the preview, but **don't save** it during the demo.

**If the live site is slow or down:** say *"the free cloud server is waking up"* and use the screenshots on slides 20–23. Everything shown there is the same system.

## Questions the external examiner may ask, with answers

**Q1. How is this different from Google Calendar or a booking form?**

A calendar only stores times. SmartDock knows **dock capacity, dock type and hazmat rules**, guarantees no overbooking under simultaneous clicks, tracks the truck through gate and dock, detects late arrivals and no-shows automatically, and turns all of that into KPIs and carrier scores.

**Q2. How exactly do you prevent double-booking?**

Every booking runs inside a database transaction that first **writes to the dock's record**. Two simultaneous bookings for the same dock then conflict; the database rejects one and retries it, and the retry sees the slot is full. I proved it: 40 simultaneous requests on two servers gave exactly 1 booking, versus 15 without the protection.

**Q3. Why MongoDB and not MySQL?**

Bookings carry flexible nested data (reschedule history, maintenance windows, operating hours). MongoDB Atlas offers a free managed cloud cluster with **multi-document transactions** and an aggregation framework for analytics. A relational database would also work. The reservation logic is the same idea in both.

**Q4. Is the data real?**

No, and the report says so clearly. The demo data is generated to mimic a real warehouse (peaks, different carrier behaviours, realistic handling times), and the before/after result is a simulation. The system itself is real and live; a pilot at a real warehouse is the next step.

**Q5. How is the reliability score calculated?**

(On-time arrivals + half of late arrivals) ÷ (all arrivals + no-shows) × 100. "Late" means more than 15 minutes after the slot start; the threshold is configurable. Tiers: A $\geq$ 90, B $\geq$ 75, C $\geq$ 50, D below 50.

**Q6. What happens if a truck arrives early or late?**

Early or within 15 minutes counts as on time. Later than that is marked late, and the delay in minutes is recorded. If a confirmed truck hasn't arrived 60 minutes after its slot start (configurable), it becomes a no-show automatically.

**Q7. How does the waitlist work?**

When a slot is full, a carrier can join its waitlist. As soon as anyone cancels, is rejected or moves their booking, the **first compatible waiting carrier** is booked automatically (first come, first served) and notified, and the manager still approves it.

**Q8. How do you keep one company's data away from another?**

Every request carries a signed login token. The server takes the user's identity and warehouse from the token, never from the form, and checks on every request that a manager touches only their warehouse and a carrier only warehouses that approved it. Automated tests confirm a manager of warehouse B cannot see or change warehouse A's data.

**Q9. Can it handle many users / scale up?**

Yes. The server is stateless and the booking guarantee works across multiple servers (I tested with two), so more server instances can be added behind the load balancer. The database is a managed cluster that can be scaled up.

**Q10. Why is analytics a bit slow on the live site?**

The free server is in the USA while the database is in Mumbai, so every query crosses the Pacific. Moving the server to the Singapore region and caching analytics would make it much faster; booking operations are already under two seconds.

**Q11. What is the supply-chain value in money terms?**

Less truck waiting means lower detention charges and driver hours; known appointments let the warehouse roster labour to the actual workload; fewer no-shows mean better use of dock capacity; and the audit trail helps settle detention disputes.

**Q12. What would you add with more time?**

No-show prediction with machine learning, automatic gate check-in with number-plate recognition, ERP integration for purchase orders and shipment notices, and a mobile app for drivers.

**Q13. What was the hardest part?**

The concurrency guarantee. A normal "check, then save" passes simple tests but fails under simultaneous clicks, and even a database transaction alone doesn't fix it. I had to make concurrent bookings collide on the dock record and then prove it with a 40-request experiment.

**Q14. How did you test it?**

58 automated end-to-end checks against a temporary database and two servers; a deliberate "protection off" experiment to prove the test is meaningful; manual testing of every screen; and a live test against the deployed site.